

第 30 讲实验设计文档

nn.Module: 模块注册、Parameter、Buffer 与 state_dict

一、实验目的

本实验不追求模型性能，而是验证 PyTorch nn.Module 的管理机制是否与我们的概念理解一致：Layer 怎样注册、Parameter 怎样被发现、Forward 是否与手工计算一致、Parameter/Buffer/普通 Tensor 怎样进入保存与设备迁移流程。

二、研究问题 (RQ)

1. RQ1: 自定义 TinyMLP(nn.Module) 后, PyTorch 能否自动发现所有 Linear Parameter?
2. RQ2: model(x) 是否与逐层手工计算得到完全相同的数值?
3. RQ3: named_parameters() 返回的参数数量能否与手工公式一致?
4. RQ4: Parameter、Buffer、普通 Tensor 在 parameters/state_dict/model.to 中有何差别?
5. RQ5: 保存并加载 state_dict() 后, 模型输出能否恢复一致?
6. RQ6: Backward 后, 注册 Parameter 的梯度是否自动写入对应 .grad?

三、实验假设

- H1: $3 \rightarrow 4 \rightarrow 2$ TinyMLP 总参数量应为 26。
- H2: 固定 Weight/Bias 后, model(x) 与手工 Forward 最大绝对误差应为 0 或浮点误差级别。
- H3: nn.Parameter 会出现在 named_parameters() 与 state_dict(); Buffer 只出现在后者和 named_buffers(); 普通 Tensor 不自动进入二者。
- H4: 新模型加载原模型的 state_dict() 后, 同一输入的输出应与原模型一致。
- H5: loss.backward() 后, 各 Parameter 的 grad.shape 应与 Parameter 自身 Shape 相同。

四、实验对象

主模型:

TinyMLP : $3 \rightarrow 4 \rightarrow 2$

结构：

$$X \rightarrow \text{Linear}(3, 4) \rightarrow \text{ReLU} \rightarrow \text{Linear}(4, 2) \rightarrow Y$$

参数总数预测：

$$(3 \times 4 + 4) + (4 \times 2 + 2) = 26$$

五、实验 1：Module 结构与 Parameter 注册

步骤：

1. 定义 TinyMLP(nn.Module);
2. 在 `__init__()` 中注册两层 Linear 和一个 ReLU;
3. 打印 model;
4. 遍历 `named_parameters()`;
5. 记录名称、Shape、`numel()` 与 `requires_grad`;
6. 对照手工参数量 26。

TinyMLP：__init__ 注册模块，forward 定义数据流

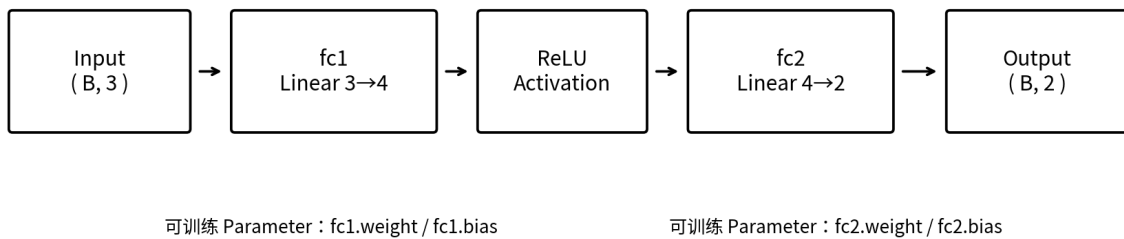


图 1: 实验模型结构

六、实验 2：Module Forward 与手工 Forward 对照

固定：

$$W_1 : (4, 3), \quad b_1 : (4,), \quad W_2 : (2, 4), \quad b_2 : (2,)$$

对同一输入分别计算：

$$\begin{aligned}
 Y_{\text{module}} &= \text{model}(X) \\
 Z_1 &= XW_1^T + b_1 \\
 A_1 &= \text{ReLU}(Z_1) \\
 Y_{\text{manual}} &= A_1W_2^T + b_2
 \end{aligned}$$

观察：

$$\max |Y_{module} - Y_{manual}|$$

应为 0 或极小浮点误差。

七、实验 3：Parameter、Buffer 与普通 Tensor

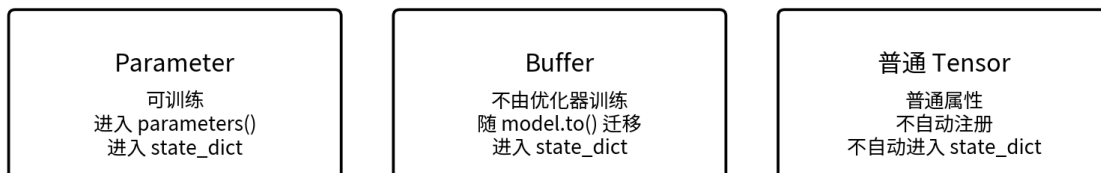
定义：

```
self.scale = nn.Parameter(torch.tensor(1.5))
self.plain_tensor = torch.tensor(2.0)
self.register_buffer("fixed_offset", torch.tensor(3.0))
```

分别检查：

- `named_parameters()`;
- `named_buffers()`;
- `state_dict().keys()`;
- `requires_grad`;
- `model.to(device)` 后的设备。

nn.Module 中三类状态的最小区别



`state_dict` = 模型可持久化状态；不是 Python 类定义，也不是 Optimizer 状态

图 2: 三类模型状态的对照

八、实验 4：state_dict 保存/恢复

步骤：

1. 获取原模型输出；
2. 保存 `model.state_dict()`;

3. 新建一个随机初始化的相同结构模型；
4. 比较加载前输出；
5. 加载 `state_dict`；
6. 再次比较输出。

判定标准：

$$\max |Y_{loaded} - Y_{original}| = 0$$

或处于浮点误差级别。

九、实验 5：Parameter 梯度自动落入.grad

对 TinyMLP：

```
model.zero_grad()
pred = model(x)
loss = pred.sum()
loss.backward()
```

逐个记录：

- Parameter 名称；
- `parameter.shape`；
- `parameter.grad.shape`；
- Gradient norm。

预期：

$$grad.shape = parameter.shape$$

十、控制变量与可复现性

项目	设置
随机性	<code>torch.manual_seed(30)</code>
主 Forward 对照	使用固定手工 Weight/Bias
输入	固定同一 Tensor
保存/加载	相同 TinyMLP 结构
数值类型	<code>float32</code>
设备	自动检测 CPU/CUDA；本次验证环境为 CPU

十一、实际结果摘要

本次运行得到：

- TinyMLP 总参数量：26；

- `model(x)` 与手工 Forward 最大绝对误差：0；
- Parameter: `scale`; Buffer: `fixed_offset`; 普通 Tensor 不进入 `state_dict`；
- 加载 `state_dict` 后，与原模型输出最大绝对误差：0；
- 所有注册 Parameter 在 backward 后都获得与自身 Shape 对齐的 `grad`。

【实验边界】 本实验验证的是 Module 组织和状态管理机制，不是在证明模型性能。科研中如果某个自定义模块没有正确注册 Parameter，训练结果可能完全失真，因此结构检查是训练前的低成本 sanity check。

十二、建议的实验记录模板

以后检查自定义模型，可以固定记录：

```
print(model)
for n, p in model.named_parameters():
    print(n, p.shape, p.requires_grad)
print(model.state_dict().keys())

# 一次 forward/backward 后
for n, p in model.named_parameters():
    print(n, p.grad is None,
          None if p.grad is None else p.grad.norm())
```

这能快速发现：参数没注册、参数没参与 Loss、梯度路径被切断等问题。